

Homework 5

Daniel Nkunga

dnkunga@uark.edu | 100292653

1. Describe the purpose of the homework and what you did / what you implemented.

The purpose of this assignment was to implement three miscellaneous algorithms focusing on string analysis, dynamic programming, and NP-completeness. The algorithm for string analysis was the Rabin-Karp, the algorithm for dynamic programming was Floyd-Warshall, and the algorithm for NP-completeness was the traveling salesperson algorithm.

2. Describe your approach and explain how you solved the problem (the key ideas for implementing each required function).

To implement Rabin-Karp, the first step is computing a hash value for the pattern and for the first m -length window of the text, where m is the length of the pattern. From there, the algorithm slides the window one character at a time across the text, updating the hash in constant time using a rolling hash function rather than recomputing it from scratch. Any time the hash of the current window matches the hash of the pattern, a character-by-character comparison is done to confirm the match and rule out hash collisions. Every confirmed match position gets added to the results vector that the function returns.

Floyd-Warshall works by maintaining a 2D distance matrix initialized from the input adjacency matrix. The core idea is to iteratively consider every vertex k as a potential intermediate stop between every pair of vertices i and j . For each combination, if the path from i to k combined with the path from k to j is shorter than the currently known direct distance from i to j , the matrix gets updated. After running through all possible intermediate vertices, the matrix holds the shortest distances between every pair of nodes in the graph. The time complexity is $O(V^3)$, which is acceptable for the campus-scale graphs being used here.

The TSP implementation uses dynamic programming with bitmasking. A bitmask is used to represent which cities have already been visited, so each unique state in the DP is defined by the current position and the set of visited cities encoded as an integer. The function is implemented recursively with memoization: $dp[mask][pos]$ stores the minimum cost to visit all remaining cities from position pos given that the cities in $mask$ have already been visited. The base case is when all cities have been visited ($mask$ equals all bits set), at which point the cost of returning to the origin is added. This approach reduces the complexity from $O(n!)$ down to $O(2^n \cdot n^2)$, which makes it feasible for small sets of locations like a campus tour.

3. Discuss any bugs or problems you encountered and how you fixed them.

The assignment went pretty smoothly! Major issues came when developing the hashing function for the Robin-Karp function because I didn't have much practice implementing them for previous classes. The traveling salesman problem was also a bit intense because I have a much easier time understanding it conceptually, but a harder time translating that understanding into working code when the problem isn't abstracted.

4. Include screenshots or images of your results.

```
[daniel_n@akihayakawa HW5_sourceCode]$ make
g++ -std=c++11 -I./include src/linked_list.cpp src/queue.cpp src/graph.cpp src/algorithms.cpp src/visualization.cpp src/main.cpp -o ./bin/main
./bin/main

=====
ALGORITHM 1: SMART CAMPUS DIRECTORY SEARCH
=====
Action: Scanning campus database for: [HILL]
Method: Rabin-Karp Rolling Hash (Digital Fingerprinting)
-----
STATUS: Success! Building identified.
Location: Found 'HILL' at position 10 in the record.
Visual: BELL_JBHT_HILL_WJWH_HAPG
      AAAA
-----

=====
ALGORITHM 2: CAMPUS-WIDE NAVIGATION CALCULATOR
=====
Status: Analyzing all possible walking routes...
Success: Shortest paths between all buildings calculated.

How to read the table below:
- The numbers represent the minimum number of 'steps' between buildings.
- 'INF' means there is currently no connected path between those two buildings.
-----
FROM \ TO      BELL  JBHT  HILL  WJWH  HAPG
-----
BELL |          0    2    1    2    3
JBHT |          2    0    1    2    3
HILL |          1    1    0    1    2
WJWH |          2    2    1    0    1
HAPG |          3    3    2    1    0
-----
Example: To get from BELL to JBHT, it takes 2 connection(s).

=====
ALGORITHM 3: EFFICIENT CAMPUS TOUR PLANNER (TSP)
=====
Mission: Visit 4 locations and return to start with minimum walking.
Target Stops: BELL -> JBHT -> HILL -> WJWH -> [Return]
Status: Solving the 'Traveling Salesperson' puzzle...
-----
RESULT: Optimal route identified.
Total Walking Distance: 6 connections.
Efficiency: This is the shortest possible path out of all 4! potential combinations.
-----
OpenCV not enabled. Run 'make OPENCV=1' to see the map.
[daniel_n@akihayakawa HW5_sourceCode]$
```